

Course on Knowledge Graphs in the era of Large Language Models

Laboratory 5: SPARQL and GraphDB

Ernesto Jiménez-Ruiz

March 26, 2026

Contents

1	Git Repository	2
2	SPARQL Playground	2
3	DBpedia knowledge graph	4
4	GraphDB tutorial	4
4.1	Installation	5
4.2	Creating repositories	6
4.3	Loading data	7
4.3.1	User interface	7
4.3.2	For Python programmers	8
4.4	Querying the data	9
4.4.1	User interface	10
4.4.2	For Python programmers	10
4.5	GraphDB with named graphs (for programmers)	11
5	Visualising Ontologies with WebVOWL	11
6	Solutions	12
6.1	Task 1: SPARQL Playground	12
6.2	Task 3: DBpedia KG	14

1 Git Repository

Support codes for the laboratory sessions are available in *github*. There is always a `requirements.txt` within each lab folder. It is recommended to use environments, but it is not strictly necessary.

<https://github.com/city-knowledge-graphs/kg-course-valgrai>

2 SPARQL Playground

We are using the SPARQL Playground dataset, a very simple data to learn SPARQL developed by researchers from the Swiss Institute of Bioinformatics <https://www.sib.swiss/>.

The SPARQL Playground is a very intuitive dataset to practice with both simple and sophisticated queries. Figure 1 shows a simplified version of the data and ontology. The same environment has also been used over more complex scenarios to understand the neXtProt and UniProt (knowledge bases about proteins) RDF models.

Task 1: Create SPARQL queries according to the descriptions below (Queries 1 to 7).

For programmers: Test them programmatically in Python. Use the `playground.ttl` data, and the codes in `queryLocalRDFGraph_Playground.py` from the GitHub repository (`lab-session-5`) as example.

For non-programmers: Test them from Protégé following these steps:

- Go to the menu *File/Check for Plugins.....*. Select and Install *Snap SPARQL*.¹
- Restart Protégé.
- Go to the menu *"Window/Create new tab..."*. Name it *Snap SPARQL*.
- Go to the menu *Window/Views/Query views* and select *Snap SPARQL Query*.
- Place it on the created Tab.
- Load in Protégé the file `playground.ttl` (available in GitHub, `python/lab-session-5/data`).
- Make sure that, if a reasoner has been selected, it has also been started (*i.e.*, reasoning has been completed).
- In the same folder, there is a file `query.txt` with an example query you can try.

Query 1 Query to return Eve's grandfather.

Query 2 Things that are dogs with color and sex. (Tip: give a look to the data in `playground.ttl`)

¹Note that the default SPARQL Protégé plugin does not work very well.

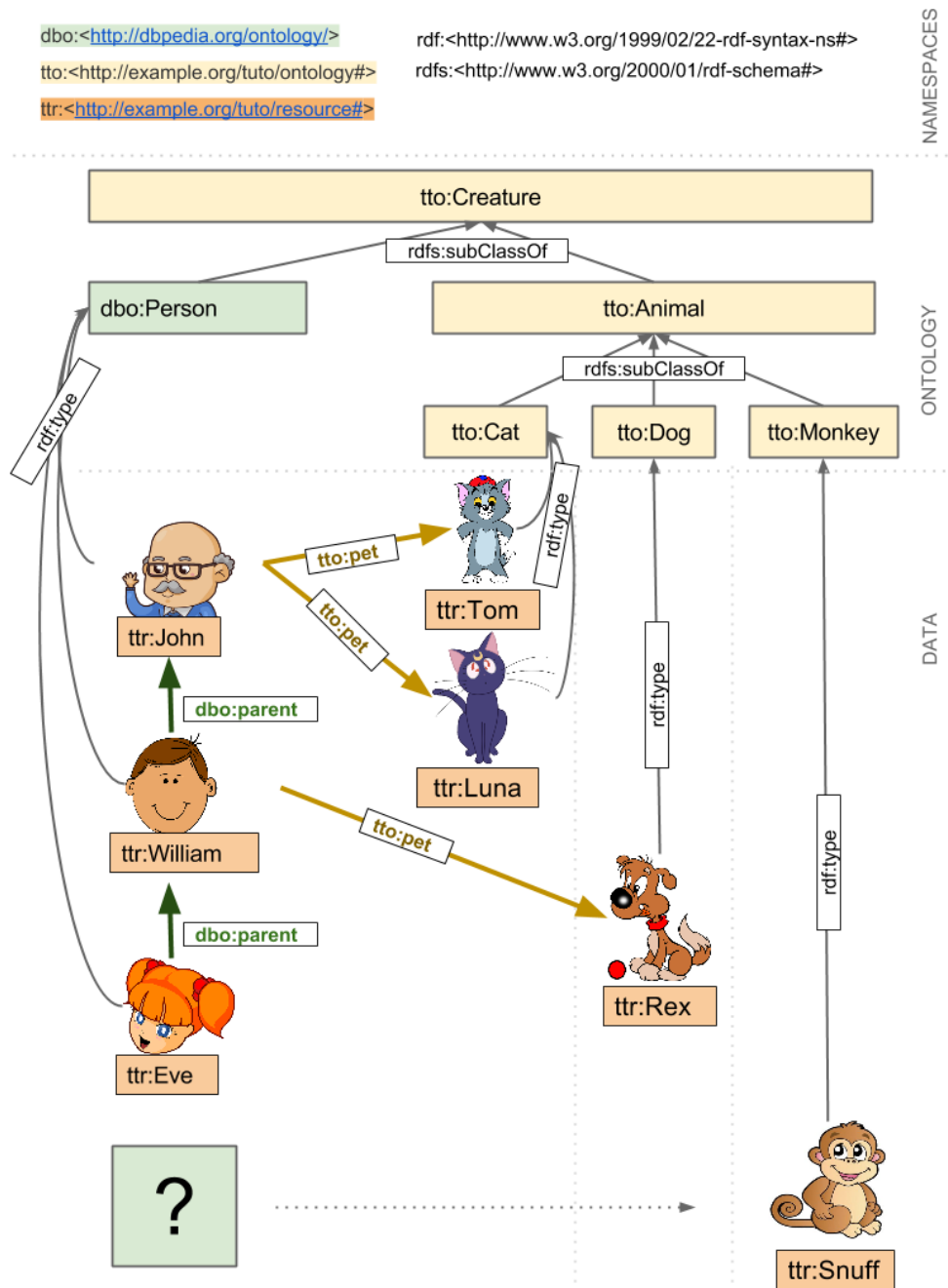


Figure 1: Simplified diagram of the data and “ontology” (from SPARQL Playground).

Query 3 Query that shows pets with their owners (Tip: owner may not exist, *i.e.*, it is optional)

Query 4 Select people with their gender and birth date ordered by gender and birth date (oldest first).

Query 5 Get the number of people by sex.

Query 6 Select persons that DO NOT have any pets

Query 7 For each pet species get the number of pets and their average weight.

3 DBpedia knowledge graph

During the lecture we have seen several example queries about “Johnny Depp”. These queries are based on the *DBpedia* Knowledge Graph, a RDF version of Wikipedia (<https://dbpedia.org/>), and can be tested via its SPARQL Endpoint (<http://dbpedia.org/sparql>).²

For programmers: Use `querySPARQLEndpoint.py` from the GitHub repository as a reference.

For non-programmers: Use the DBpedia’s Web interface: <http://dbpedia.org/sparql>.

Task 2: Try (some of) the queries in the lecture slides.

Task 3: Modify the running example query from the lecture such that it retrieves performers co-starring with Johnny Depp that were born in Spain.

Tip: explore https://dbpedia.org/page/Pen%C3%A9lope_Cruz to see which predicates you may need in the query.

4 GraphDB tutorial

Libraries such as RDFLib (Python) are useful for managing RDF graphs, but for storing, reasoning, and querying large knowledge graphs, we need better backend solutions. RDFLib will continue to be used locally, for example, to create triples and to communicate with a graph database via its SPARQL Endpoint.

In this lab we are using the graph database GraphDB³ developed by Ontotext (<https://www.ontotext.com/>) which has a set of interesting characteristics: (i) free version with a large set of features, (ii) compliant with Semantic Web standards, and (iii) with tutorials and documentation.⁴

²Access point for the dataset via SPARQL.

³The screenshots are based on GraphDB version 10.8 (as of April 2025).

⁴<https://www.ontotext.com/knowledge-hub/>

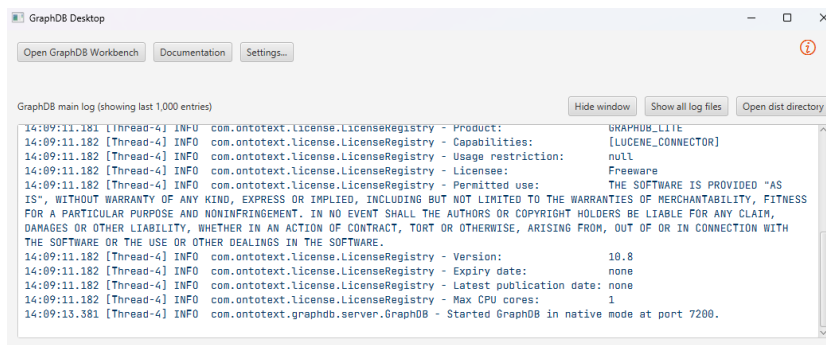


Figure 2: GraphDB initial window.

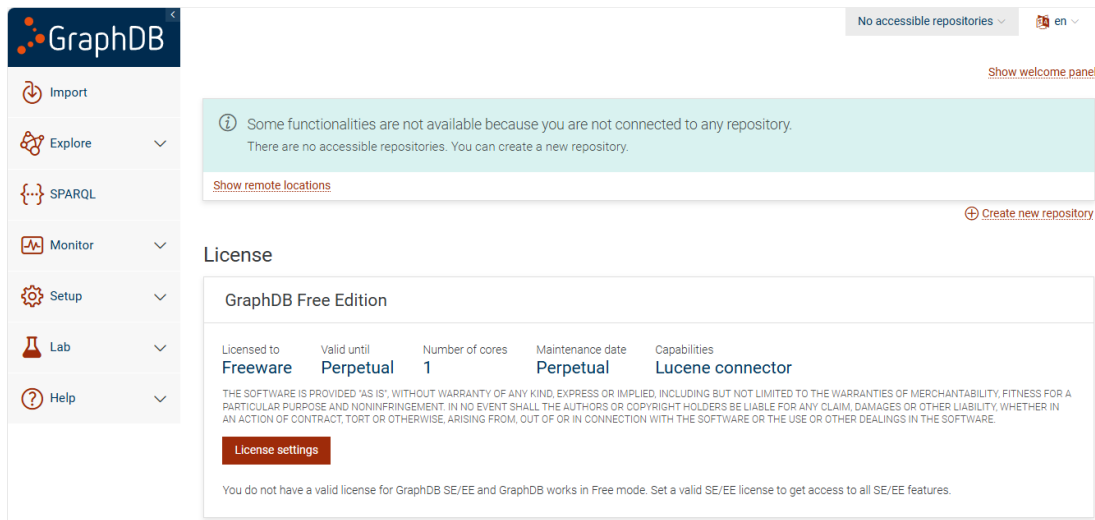


Figure 3: GraphDB welcome screen.

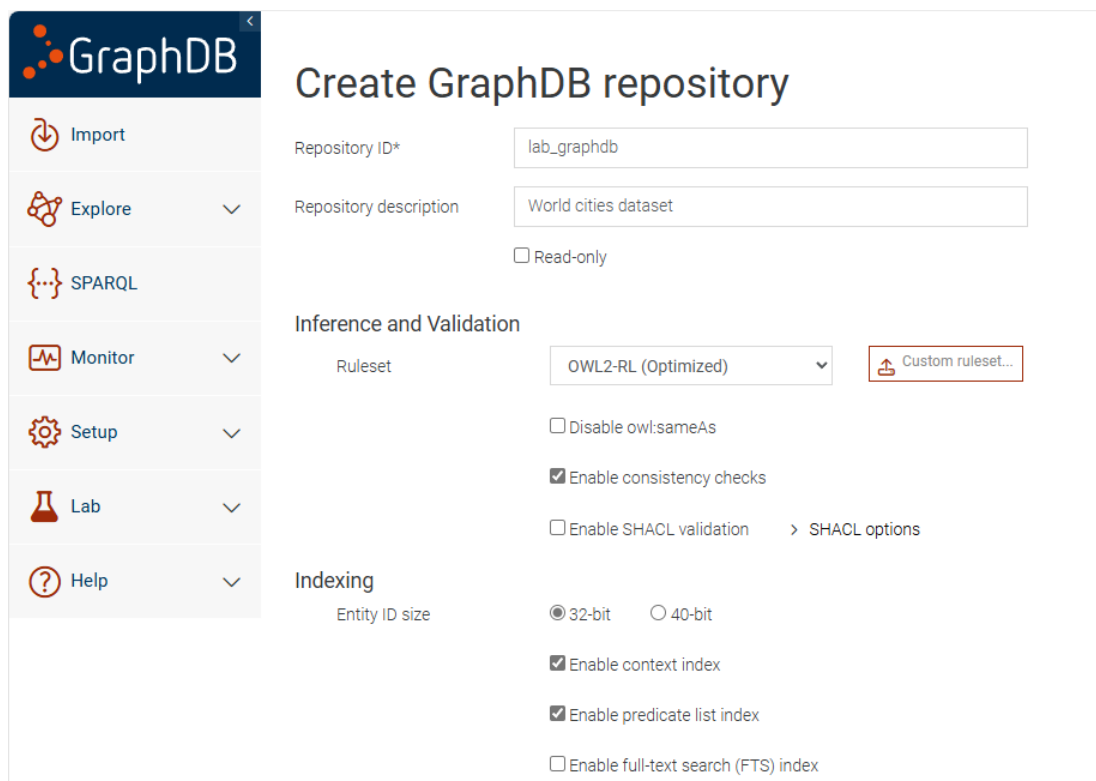
4.1 Installation

GraphDB is available for basically all operating systems. To download the free version, one needs to fill a form to get a free license: <https://www.ontotext.com/products/graphdb/>.

For convenience we are using GraphDB as a desktop application. GraphDB can also be run as a command-line standalone server which is the standard way of using GraphDB in production. The Getting Started Guide (<https://graphdb.ontotext.com/documentation/10.8/>) shows how to install and run GraphDB as a desktop application in the different platforms.⁵ Once GraphDB has been launched via its Desktop application a window like the one in Figure 2 will appear. From this window, you can change the settings of the GraphDB server (*e.g.*, the port, the default is 7200), access the online documentation, or open GraphDB.⁶ GraphDB opens via

⁵The desktop client, apart from running GraphDB as a server, also provides a user interface. In principle, the desktop version is self-contained, but you may need to install Java: <https://www.java.com/en/download/>.

⁶Note that if port 7200 is already being used by another application, you will need to choose a different one.



GraphDB

Create GraphDB repository

Repository ID*

Repository description

☐ Read-only

Inference and Validation

Ruleset [Custom ruleset...](#)

☐ Disable owl:sameAs

☒ Enable consistency checks

☐ Enable SHACL validation > SHACL options

Indexing

Entity ID size ☒ 32-bit ☐ 40-bit

☒ Enable context index

☒ Enable predicate list index

☐ Enable full-text search (FTS) index

Figure 4: GraphDB repository creation.

the (default) Web browser with the local URL `http://localhost:7200/`. You can use your favourite Web browser by just copying and pasting this GraphDB URL.

4.2 Creating repositories

The first step to start using GraphDB is the creation of a GraphDB repository (see welcome screen in Figure 3). Repositories can be created with customised characteristics (see Figure 4). In our setting, the most important ones are: (i) disabling Read-only (default), (ii) enabling OWL 2 RL reasoning from the list of supported languages, and (iii) enabling the consistency checks.

The created repository acts as a SPARQL Endpoint and can be accessed as such. Figure 5 shows how to access the URL of the created repository. For example:

```
'http://127.0.0.1:7200/repositories/lab_graphdb'
```

is the URL of the repository that has been created. It can be accessed programmatically as any other SPARQL Endpoint, as we covered during the SPARQL laboratory sessions.

You can create several GraphDB repositories, although only one can be the *active* repository to be used with the desktop application (see GraphDB repositories view in Figure 6). Programmatically, any of the repositories can be accessed, as long as GraphDB has been launched and the repositories are RUNNING (see Figure 6).



Figure 5: URL of the active repository in GraphDB (e.g., URL of the SPARQL Endpoint).

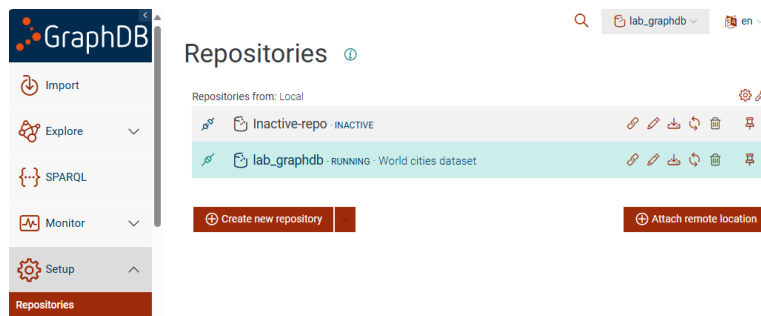


Figure 6: GraphDB repositories view.

4.3 Loading data

We can load data to a GraphDB repository via both the user interface and programmatically. We will use as an example the World cities 5 ★ data and ontology created in the previous laboratory session. The files can be found in the GitHub repository (*python/lab-session5/graphdb/data*):

- Ontology: `ontology_worldcities.ttl`
- Data: `worldcities-free-100-task5stars.ttl`

4.3.1 User interface

GraphDB allows to upload local and remote files in different formats including `.ttl` and `.owl` (see Figure 7). Once the files have been uploaded one need to import each of them into the repository (button *import*). You will be asked for a base URI, leave empty (will use the default in your data/ontology) or indicate a new one. You will also be asked to add triples to the default graph or create a specific named graph.

One can also import RDF data and an ontology from a remote URL. For example you can use:

- Ontology: `https://raw.githubusercontent.com/city-knowledge-graphs/kg-course-valgrai/refs/heads/main/python/lab-session5/graphdb/data/ontology_worldcities.ttl`

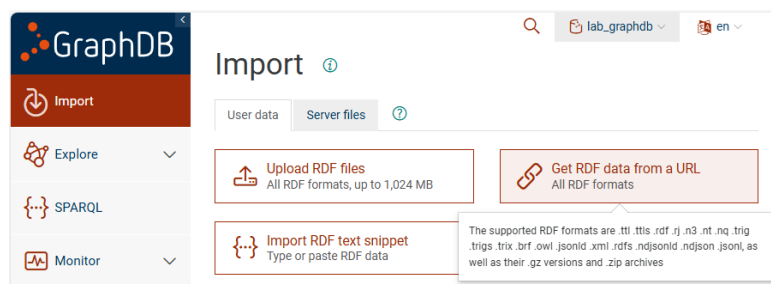


Figure 7: GraphDB upload and import RDF graphs and ontologies from files.

	subject	predicate	object	context
1	wco:delhi	wco:admin_name	"Delhi"	http://www.ontotext.com/ex...
2	wco:delhi	wco:isFirstLevelAdminCapit...	wco:india	http://www.ontotext.com/ex...
3	wco:delhi	wco:latitude	"28.66" ^{xsd:float}	http://www.ontotext.com/ex...
4	wco:delhi	wco:longitude	"77.23" ^{xsd:float}	http://www.ontotext.com/ex...
5	wco:delhi	wco:name	"Delhi"	http://www.ontotext.com/ex...
6	wco:delhi	wco:name_ascii	"Delhi"	http://www.ontotext.com/ex...
7	wco:delhi	wco:population	"29617000" ^{xsd:long}	http://www.ontotext.com/ex...
8	wco:delhi	rdf:type	wco:City	http://www.ontotext.com/ex...

Figure 8: GraphDB RDF graph exploration (Tabular format).

- **Data:** <https://raw.githubusercontent.com/city-knowledge-graphs/kg-course-valgrai/refs/heads/main/python/lab-session5/graphdb/data/worldcities-free-100-task5stars.ttl>

Once the local files and/or remote resources have been loaded, you can explore the loaded triples in a tabular (see Figure 8) or graphical form (see Figure 9), and via SPARQL queries (see Section 4.4). The data can also be cleared from the repository in the 'Graphs overview' menu.

4.3.2 For Python programmers

There are several options to upload data programmatically.

- **SPARQL Update:**⁷ using the SPARQL Update language may be useful to perform minor updates (*e.g.*, load individual triples).
- **ImportRDF tool:**⁸ this is the best option for efficient (offline) upload of large amounts of data.

⁷<https://www.w3.org/TR/sparql11-update/>

⁸<https://graphdb.ontotext.com/documentation/10.8/loading-data-using-importrdf.html>

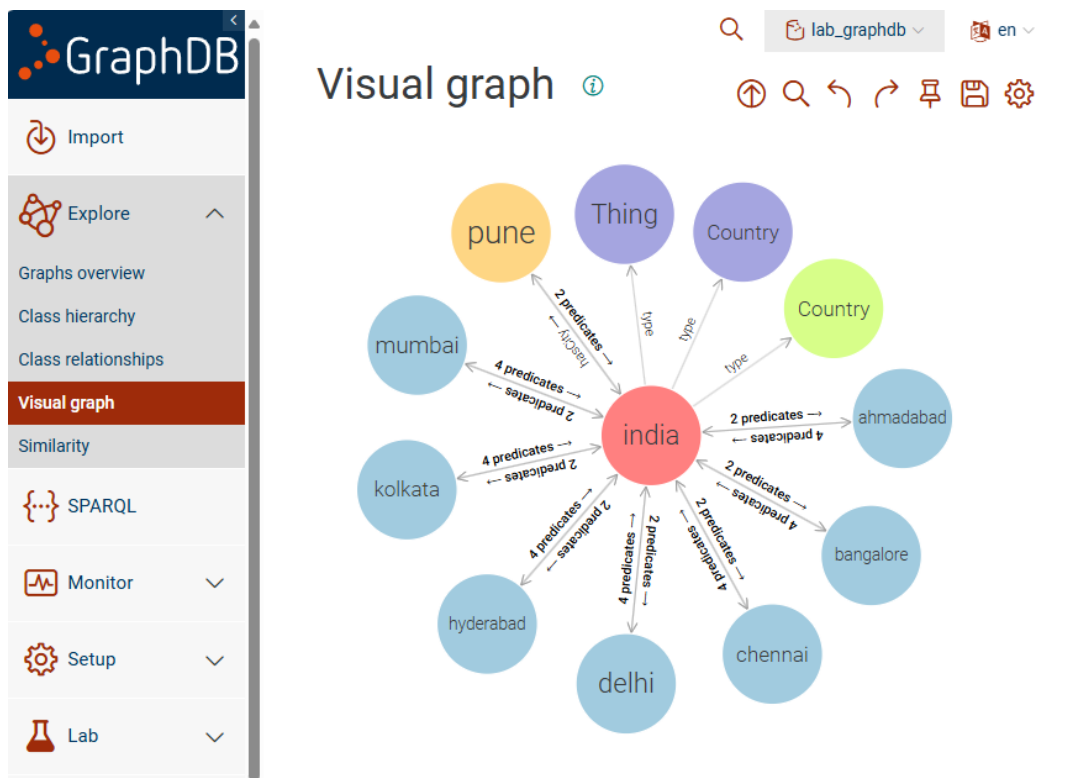


Figure 9: GraphDB RDF graph exploration (graphical format).

- **HTTP client request:** this is the option we will adopt in this lab session using the commandline tool `cURL` (Client URL).⁹ `cURL` can be executed from Python. Command to be executed:

```
curl 'graphdb_upload_uri' -X POST -H
"Content-Type:application/x-turtle" -T 'datafile.ttl'
```

Where `'datafile.ttl'` is the RDF data to load and `graphdb_upload_uri` is the concatenation of the SPARQL Endpoint URL (as in Figure 5) and `'/statements'`. For example:

```
http://127.0.0.1:7200/repositories/lab_graphdb/statements
```

Support codes: The Python scripts `graphdb_communication.py`, and `graphdb_notebook.ipynb`, provide examples of how to load in and query from GraphDB.

4.4 Querying the data

We can query data from a GraphDB repository using SPARQL queries via both the user interface and programmatically.

⁹Windows users may need to install `cURL`: <https://curl.se/windows/>

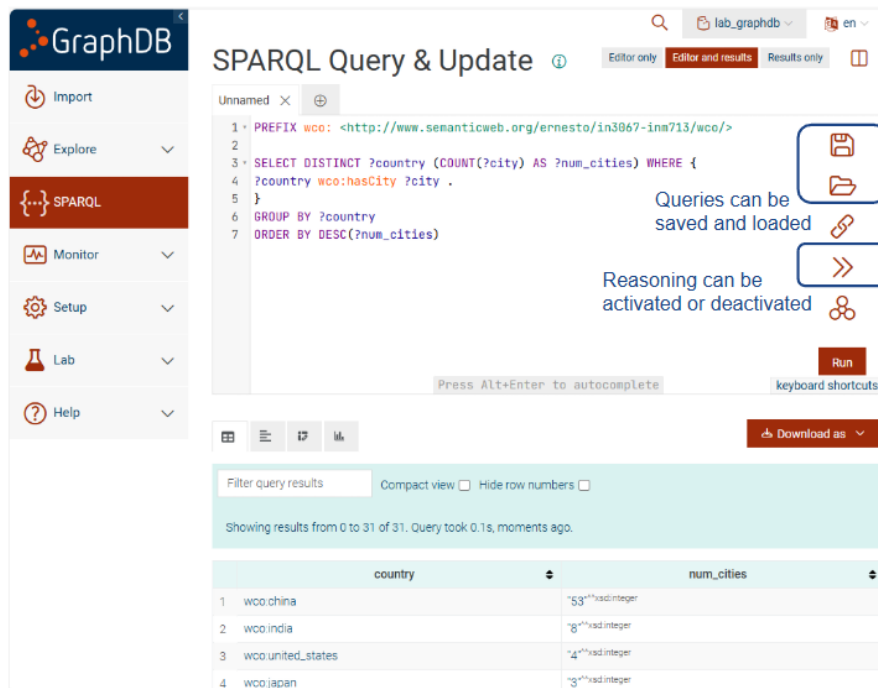


Figure 10: GraphDB query interface.

4.4.1 User interface

Figure 10 shows a query that counts the cities in each country and gives the output ordered by number of cities. Using the GraphDB interface, (i) queries can be executed, (ii) queries can be named, saved and loaded, and (iii) query results can be downloaded in a number of formats (e.g., CSV or JSON).

```
PREFIX wco: <http://www.semanticweb.org/ernesto/in3067-inm713/wco/>

SELECT DISTINCT ?country (COUNT(?city) AS ?num_cities) WHERE {
    ?country wco:hasCity ?city .
}
GROUP BY ?country
ORDER BY DESC(?num_cities)
```

4.4.2 For Python programmers

The GraphDB repositories can be accessed and queried as a standard SPARQL Endpoint.

Support codes: Similarly to the solutions to the SPARQL Lab sessions, the scripts `graphdb_communication.py`, `graphdb_notebook.ipynb` and `GraphdbCommunication.java` include an example to execute the above query over a GraphDB repository via its SPARQL Endpoint.

4.5 GraphDB with named graphs (for programmers)

RDFlib does not provide clear support for named graphs. GraphDB, however, implements this functionality.¹⁰

Task 4. Create a new GraphDB repository to store the named graphs in `named_graphs.ttl` (file available in GitHub). Note that the format of the file is *TriG* as regular Turtle do not support named graphs. Extend the `named_graphs.ttl` dataset to include triples about yourself in the correspondent named graph. Use the scripts

`graphdb_communication.py` and `GraphdbCommunication.java` as examples to load and query the named graphs.

5 Visualising Ontologies with WebVOWL

WebVOWL (<https://github.com/VisualDataWeb/WebVOWL>) is an interactive, web-based visualisation tool for exploring OWL ontologies as graphs (see Figure 11). It transforms ontology files into intuitive visual representations of classes, properties, and their relationships, allowing users to inspect, filter, and navigate complex knowledge structures through zooming and interactive node selection.

Like other visualisation tools, it has limitations:

- Best for small to medium-sized ontologies (large ones can get cluttered).
- It focuses on the ontology structure, not instance data.
- It is a viewer, not an editing tool like Protégé.

¹⁰<https://graphdb.ontotext.com/documentation/10.8/sparql-compliance.html>

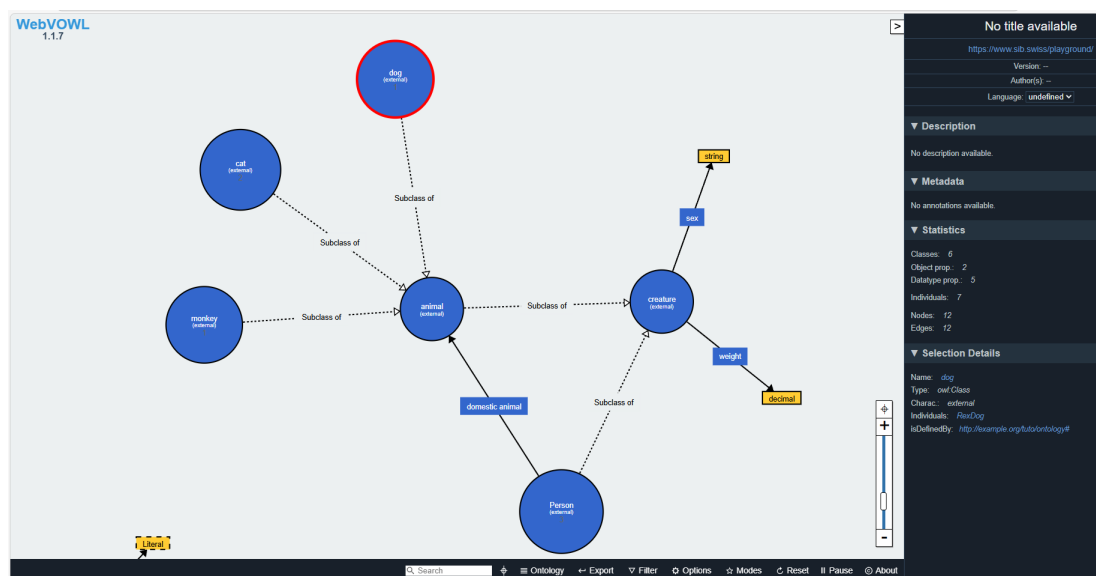


Figure 11: The *playground.ttl* ontology in WebVOWL.

Task 5. Visualise in WebVOWL the ontologies we mentioned in this lab session. Use the available Web service: <https://service.tib.eu/webvowl/>. In the menu "Ontology" (bottom), you can upload a local ontology or indicate the URI of a remote ontology (e.g., <https://city-knowledge-graphs.github.io/pizza-restaurants-ontology/release/1.0.0/ontology.owl>)

The URI of the ontology can also be indicated directly on the URL of the service: <https://service.tib.eu/webvowl/#iri=https://city-knowledge-graphs.github.io/pizza-restaurants-ontology/release/1.0.0/ontology.owl>

6 Solutions

6.1 Task 1: SPARQL Playground

The solutions assume the relevant prefixes have been defined in the 'prologue'. Sometimes, when running queries against an endpoint some of the prefixes are already defined on the endpoint side. In case of doubt (or error), however, it is better if the queries are self-contained with the declaration of the used prefixes. Common general prefixes:

```
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
```

Specific prefixes for the SPARQL playground queries.

```
PREFIX tto: <http://example.org/tuto/ontology#>
PREFIX ttr: <http://example.org/tuto/resource#>
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbp: <http://dbpedia.org/property/>
PREFIX dbpedia: <http://dbpedia.org/resource/>
```

Query 1: using paths

```
SELECT ?grandfather where {
    ttr:Eve dbo:parent/dbo:parent ?grandfather .
}
```

Query 1: (alternative 1 using blank node syntax)

```
SELECT ?grandfather where {
    ttr:Eve dbo:parent [ dbo:parent ?grandfather ] .
}
```

Query 1: (alternative 2 without blank node syntax)

```
SELECT ?grandfather where {
    ttr:Eve dbo:parent ?x .
}
```

```

    ?x dbo:parent ?grandfather .
}

```

Query 2:

```

SELECT ?dogs ?color ?sex WHERE {
    ?dogs rdf:type tto:Dog ;
        tto:sex ?sex;
        tto:color ?color .
}

```

Query 3:

```

SELECT ?pet ?owner where {
    ?pet a [ rdfs:subClassOf tto:Animal ].
    OPTIONAL {?owner tto:pet ?pet}
}

```

Query 3: (alternative without blank node syntax)

```

SELECT ?pet ?owner where {
    ?pet a ?petclass .
    ?petclass rdfs:subClassOf tto:Animal .
    OPTIONAL {?owner tto:pet ?pet}
}

```

Query 3: (alternative if reasoning is enabled - week 7)

```

SELECT ?pet ?owner where {
    ?pet a tto:Animal .
    OPTIONAL {?owner tto:pet ?pet}
}

```

Query 4:

```

SELECT ?people ?gender ?bd WHERE {
    ?people rdf:type dbo:Person ;
        tto:sex ?gender ;
        dbp:birthDate ?bd .
}
ORDER BY ?gender ASC(?bd)

```

Query 5:

```

SELECT DISTINCT ?sex (COUNT(?people) as ?peopleCount) WHERE {
    ?people rdf:type dbo:Person .
    ?people tto:sex ?sex .
}
GROUP BY ?sex

```

Query 6:

```
SELECT DISTINCT ?person ?pet WHERE {  
    ?person rdf:type dbo:Person .  
    FILTER NOT EXISTS {?person tto:pet ?pet } .  
}
```

Query 7:

```
SELECT DISTINCT ?species (COUNT(?pet) as ?petCount) (AVG(?weight)  
as ?avgWeight) WHERE {  
    ?species rdfs:subClassOf tto:Animal .  
    ?pet rdf:type ?species .  
    ?pet tto:weight ?weight .  
}  
GROUP BY ?species
```

6.2 Task 3: DBpedia KG

```
SELECT DISTINCT ?costar WHERE {  
    ?jd foaf:name "Johnny Depp"@en .  
    ?m dbo:starring ?jd .  
    ?m dbo:starring ?other .  
    ?other foaf:name ?costar .  
    ?other dbo:birthPlace ?place .  
    ?place dbo:country dbr:Spain  
    FILTER (STR(?costar)!="Johnny Depp")  
}  
ORDER BY ?costar
```